

PacMan em Rede com Protocolo Próprio sobre Ethernet Bruta

Arthur Dias Viana de Castro – GRR20190371

Mateus Ribamar da Paixão – GRR20190154

Universidade Federal do Paraná – UFPR
Departamento de Informática

1 Introdução

O objetivo deste trabalho foi desenvolver uma versão em rede do jogo PacMan utilizando comunicação direta entre duas máquinas Linux conectadas por cabo. Diferentemente de uma aplicação tradicional baseada em sockets TCP ou UDP, a comunicação foi realizada sobre Ethernet bruta, permitindo maior controle sobre o formato das mensagens transmitidas e sobre os mecanismos de confiabilidade implementados.

Essa abordagem permitiu exercitar, de forma prática, conceitos fundamentais de redes de computadores, como enquadramento, endereçamento em nível de enlace, confirmação de recebimento, retransmissão, fragmentação de mensagens, janela deslizando e detecção de erros. O código-fonte do projeto está disponível no repositório: <https://github.com/Ribapax/tIredesI2601>.

2 Implementação

A aplicação foi organizada em camadas, separando a lógica do jogo, o protocolo de comunicação, o transporte em Ethernet bruta e a coordenação da sessão cliente-servidor.

O ponto de entrada do programa fica em `src/main.c`. Esse módulo carrega a configuração inicial, escolhe o papel da máquina, monta a mensagem inicial INIT e inicia o processo de handshake. A configuração é lida a partir de um arquivo texto e inclui o modo de jogo, o papel inicial da máquina, o tamanho da janela de transferência, o tamanho inicial da visualização, o mapa utilizado e a interface de rede.

A lógica de domínio do jogo fica isolada em `src/domain/`, incluindo mapa, movimentação do PacMan, movimentação dos fantasmas e regras de visibilidade. A sessão cliente-servidor fica em `src/app/game_session.c`, onde são coordenados os movimentos, mensagens ACK e NACK, envio de VIEW, transferência de arquivos, timeouts, retransmissões e encerramento da sessão.

3 Protocolo de Comunicação

O protocolo utiliza frames próprios com marcador `0x7e`, payload de até 31 bytes, número de sequência de 0 a 63, tipo de mensagem e CRC-8. A serialização dos frames fica em

`src/protocol/frame_codec.c`, enquanto as estruturas e tipos principais são definidos em `include/net/frame.h`.

O handshake inicial é feito diretamente sobre Ethernet bruta. O servidor envia uma mensagem **INIT** em broadcast, o cliente responde com **ACK** e, a partir desse momento, o servidor passa a utilizar comunicação unicast com o endereço MAC descoberto. O acesso a sockets `AF_PACKET` fica em `src/transport/raw_eth.c`, enquanto o handshake é implementado em `src/transport/net_handshake.c`.

No fio, todos os tipos de mensagem usam o mesmo cabeçalho. O byte 0 contém o marcador `0x7e`; o byte 1 contém `length[4:0]` e `sequence[5:3]`; o byte 2 contém `sequence[2:0]` e `type[4:0]`; os bytes seguintes carregam o payload de 0 a 31 bytes; e o último byte contém o CRC-8 calculado sobre o cabeçalho e o payload.

Tabela 1: Tipos de mensagem do protocolo.

Tipo	Código	Payload
ACK	0	Vazio; a sequência confirma o frame recebido.
NACK	1	Vazio; a sequência indica o frame esperado para retransmissão.
VIEW	2	6 bytes: <code>file_id=0</code> , <code>window_size=1</code> , <code>total_bytes</code> .
INIT	3	Vazio; usado no handshake inicial por broadcast.
DATA	4	Até 31 bytes brutos de uma transferência.
FILE_TXT	5	6 bytes: <code>file_id</code> , <code>window_size</code> , <code>total_bytes</code> .
FILE_JPG	6	6 bytes: <code>file_id</code> , <code>window_size</code> , <code>total_bytes</code> .
FILE_MP4	7	6 bytes: <code>file_id</code> , <code>window_size</code> , <code>total_bytes</code> .
MOVE_RIGHT	10	Vazio; comando de movimento para direita.
MOVE_LEFT	11	Vazio; comando de movimento para esquerda.
MOVE_UP	12	Vazio; comando de movimento para cima.
MOVE_DOWN	13	Vazio; comando de movimento para baixo.
ERROR	15	3 bytes: código de erro, tipo relacionado e <code>file_id</code> .
END	16	Vazio; solicita encerramento da sessão.

Mensagens maiores que um frame são fragmentadas em frames **DATA**. A visualização do jogo no formato 40x40 é compactada com 4 bits por célula em `src/app/game_session_view_codec.c`. Os arquivos associados às pastilhas também utilizam o mesmo mecanismo de transferência. As pastilhas 1 e 2 enviam textos, as pastilhas 3 e 4 enviam imagens JPG, as pastilhas 5 e 6 enviam vídeos MP4, e o encontro com um fantasma envia o arquivo `resources/files/ghost.txt`.

A confiabilidade da transferência é controlada por janela deslizante em `src/reliable/window.c`. O mecanismo implementa confirmações cumulativas, tratamento de mensagens duplicadas e entrega em ordem. Erros irrecuperáveis utilizam mensagens do tipo **ERROR**, separadas de **NACK**. Essa separação é necessária porque **NACK** indica que o receptor espera uma retransmissão, enquanto **ERROR** indica que a transferência corrente deve ser abortada com um motivo específico.

4 Testes e Resultados

A validação foi realizada por meio de testes manuais voltados ao comportamento do protocolo de comunicação entre cliente e servidor. Foram testados timeout, retransmissão,

duplicidade de mensagens, falha física de conexão e tratamento de erros no envio de arquivos.

No teste de timeout, verificou-se que, quando uma mensagem não recebe confirmação, o emissor retransmite a mensagem. O tempo de espera aumenta progressivamente a cada cinco ocorrências de **NACK** ou timeout, até o limite máximo de 5 segundos.

No teste de mensagens duplicadas, verificou-se que uma mesma mensagem recebida mais de uma vez é processada apenas uma vez. Para isso, o protocolo mantém uma lista de mensagens já recebidas, impedindo que o mesmo comando, como um movimento do PacMan, seja aplicado repetidamente.

No teste de falha física de conexão, o cabo de rede foi removido durante a execução. Enquanto a conexão permanece indisponível, o programa continua tentando transmitir as mensagens pendentes. Quando a conexão é restabelecida, a comunicação pode continuar a partir das retransmissões.

O envio de arquivos foi testado durante a execução do jogo, acionando as pastilhas associadas a arquivos de texto, imagem e vídeo. Todos os arquivos previstos foram enviados e testados em execução, validando a fragmentação em frames **DATA**, a transmissão e a remontagem no cliente.

Também foram testados os casos de arquivo inexistente e arquivo com nome incorreto. Nessas situações, o cliente informa que o arquivo não existe e a pastilha não é consumida. Assim, sempre que o PacMan passa novamente pela mesma posição, o servidor tenta realizar outra vez a transferência do arquivo correspondente.

Os testes indicam que o protocolo consegue lidar com perdas de resposta, retransmissões, mensagens duplicadas, falhas físicas temporárias e erros no envio de arquivos, mantendo o estado do jogo consistente durante a execução.